C++ book reading checklist

37 core reading blocks across 7 phases (phase 6 has none), in study order, plus an optional move-semantics deep dive. Page numbers are PDF page numbers (the number your PDF viewer shows), not the numbers printed on the book pages. Tick a box when you have finished the block.

Phase 1: Core language foundations

☐	**C++ Primer**: ch. 2 Variables and Basic Types (2.1 to 2.6)	PDF pp. 61 to 109
☐	**C++ Primer**: ch. 3 Strings, Vectors and Arrays (3.2 to 3.6)	PDF pp. 113 to 161
☐	**C++ Primer**: 4.2 to 4.5 Operators (short-circuit `&&`/`||`, prefix versus postfix `++`); 4.8 Bitwise operators; 4.11 Type conversions	PDF pp. 168 to 178; 181 to 184; 188 to 194
☐	**C++ Primer**: 5.3 to 5.5 Statements: `switch` fall-through, the dangling `else`, loops and jumps	PDF pp. 203 to 221
☐	**C++ Primer**: ch. 6 Functions (6.1 to 6.7)	PDF pp. 231 to 281

Phase 2: Classes, memory and object-oriented design

☐	**C++ Primer**: ch. 7 Classes (7.1 to 7.6)	PDF pp. 283 to 335
☐	**C++ Primer**: 12.1.2 Managing memory directly; 12.2.1 `new` and arrays	PDF pp. 487 to 492; 506 to 509
☐	**C++ Primer**: ch. 13 Copy Control (13.1 to 13.5)	PDF pp. 525 to 559
☐	**C++ Primer**: ch. 14 Overloaded Operations and Conversions (14.1 to 14.9)	PDF pp. 581 to 619
☐	**C++ Primer**: ch. 15 Object-Oriented Programming (15.1 to 15.9)	PDF pp. 621 to 679
☐	**C++ Primer**: 18.3 Multiple and virtual inheritance; 19.2 Run-time type identification	PDF pp. 831 to 847; 854 to 860

Phase 3: The standard library and generic programming

☐	**C++ Primer**: ch. 8 The IO Library	PDF pp. 339 to 353
☐	**C++ Primer**: ch. 9 Sequential Containers (9.1 to 9.6)	PDF pp. 355 to 403
☐	**C++ Primer**: ch. 10 Generic Algorithms (10.1 to 10.6)	PDF pp. 405 to 447
☐	**C++ Primer**: ch. 11 Associative Containers (11.1 to 11.4)	PDF pp. 449 to 477
☐	**C++ Primer**: ch. 16 Templates: 16.1 to 16.3, 16.5 (skip 16.2.5 to 16.2.7 and 16.4 until phase 5)	PDF pp. 681 to 706; 707 to 718; 723 to 727; 735 to 743
☐	**Effective Modern C++**: Items 31 to 32: lambdas	PDF pp. 234 to 247
☐	**C++ Primer**: ch. 17: 17.1 tuple; 17.3 regular expressions; 17.4 random numbers	PDF pp. 747 to 751; 757 to 773; 774 to 780

Phase 4: RAII, exceptions and advanced language tools

☐	**C++ Primer**: 5.6 `try` blocks; 18.1 Exception handling; 18.2 Namespaces	PDF pp. 222 to 228; 801 to 830
☐	**C++ Primer**: 19.1 Controlling memory allocation; 19.3 enumerations; 19.5 nested classes; 19.6 `union`	PDF pp. 849 to 853; 861 to 863; 872 to 875; 876 to 880
☐	**Programming: Principles and Practice**: RAII, exceptions and resource management: 18.4 to 18.5	PDF pp. 542 to 553

Phase 5: Modern C++ (C++11 to C++20)

☐	**C++ Primer**: 12.1 smart pointers (shared_ptr, unique_ptr, weak_ptr)	PDF pp. 479 to 486; 493 to 504
☐	**C++ Primer**: 13.6 Moving objects	PDF pp. 560 to 579
☐	**C++ Primer**: 16.2.5 to 16.2.7 deduction, `std::move`, forwarding; 16.4 variadic templates	PDF pp. 716 to 722; 728 to 734
☐	**Effective Modern C++**: Items 1, 2, 5, 7: deduction, `auto`, `()` versus `{}`	PDF pp. 27 to 41; 55 to 61; 67 to 76
☐	**Effective Modern C++**: Item 15 `constexpr`	PDF pp. 115 to 121
☐	**Effective Modern C++**: Items 18 to 22: `unique_ptr`, `shared_ptr`, `weak_ptr`, `make_*`, Pimpl	PDF pp. 136 to 174
☐	**Effective Modern C++**: Items 23, 24, 25, 28: `move`, `forward`, universal references, reference collapsing	PDF pp. 176 to 195; 215 to 221
☐	**Effective Modern C++**: Items 26 to 27, 29, 41, 42: overloading, cheap moves, pass by value, emplacement	PDF pp. 195 to 215; 221 to 225; 299 to 320

Phase 7: Concurrency and the memory model

☐	**C++ Concurrency in Action**: Why threads and launching them: 1.1, 1.2.3, 2.1, 2.2	PDF pp. 25 to 30; 32 to 33; 40 to 50
☐	**C++ Concurrency in Action**: Sharing data and mutexes: 3.1, 3.2.1 to 3.2.3	PDF pp. 60 to 74
☐	**C++ Concurrency in Action**: Deadlock and locking: 3.2.4 to 3.2.8, 3.3.1	PDF pp. 74 to 91
☐	**C++ Concurrency in Action**: Waiting for events and results: 4.1, 4.2.1 to 4.2.4	PDF pp. 96 to 113
☐	**C++ Concurrency in Action**: Thread counts and a pool: 2.4, 9.1.1	PDF pp. 54 to 57; 324 to 326
☐	**C++ Concurrency in Action**: The memory model and data structures: 5.1, 5.3 overview, 6.1 to 6.2.2	PDF pp. 148 to 151; 165 to 170; 197 to 206
☐	**C++ Concurrency in Action**: Performance and bugs: 8.2, 11.1 to 11.2.1	PDF pp. 283 to 289; 363 to 367

Phase 8: Performance, memory layout and binary data

☐	**C++ Primer**: 17.2 `bitset`; 19.8 bit-fields, `volatile`, `extern "C"`	PDF pp. 752 to 756; 883 to 892

Optional: move semantics deep dive

☐	Meyers Item 17, special member function generation (PDF pages 127 to 133)
☐	Meyers Items 24 and 28, universal references and reference collapsing (PDF pages 182 to 186 and 215 to 221)
☐	Josuttis chapter 2, core features including moved-from objects (PDF pages 43 to 52)
☐	Josuttis 3.1 and 3.3 to 3.4, the rules for special member functions and the Rule of Five or Three (PDF pages 53 to 58 and 66 to 76)
☐	Josuttis 6.1 to 6.2, moved-from states (PDF pages 107 to 115)
☐	Josuttis 7.1 and 7.4, `noexcept` and when to use it (PDF pages 125 to 134 and 140 to 141)
☐	Josuttis 8.1.2 to 8.1.3 and 8.4 to 8.5, value categories (PDF pages 145 to 148 and 153 to 154)
☐	Josuttis 9.1 to 9.3, perfect forwarding and universal references (PDF pages 159 to 166)
☐	Self-check from memory: write a class with a `noexcept` move constructor and move assignment, say what state its moved-from object is in, and explain the `const` trap and why `return std::move(local)` is a mistake

From the cpp-training study checklist. Interactive version with saved progress: https://ghaith-hamdi1.github.io/cpp-training/study-checklist.html